

Assignment 2.2: Search Lab — Shortest Driving Route in San Francisco

AAI-501 · Module 2 · Gunasekaran Pasupathy · 6/4/2026 - 6/6/2026

Problem: We have to find the shortest driving route between **Oracle Park** (the SF Giants stadium) and the **Palace of Fine Arts** in San Francisco.

My approach: First I will load the real road network of San Francisco from OpenStreetMap using the **OSMnx** library. This gives us a graph where intersections are nodes and roads are edges, and each edge has a length in meters. Then I will find the shortest path using two search algorithms from **NetworkX**: **Dijkstra's algorithm** and **A* search**. This is basically how GPS apps like Google Maps work internally.

```
In [1]: %pip install osmnx networkx matplotlib scikit-learn --quiet
```

Note: you may need to restart the kernel to use updated packages.

```
In [2]: import networkx as nx
import osmnx as ox
import matplotlib.pyplot as plt
import time

ox.settings.use_cache = True #Cache OSM API responses locally so re-runs are fast
print("OSMnx version:", ox.__version__)
```

OSMnx version: 2.0.7

Step 1: Get the coordinates of both places

First we need the latitude and longitude of both locations. OSMnx has a geocode function which uses OpenStreetMap's Nominatim service, so we can just give the place name and get the coordinates.

```
In [3]: origin_point = ox.geocode("Oracle Park, San Francisco, CA")
dest_point = ox.geocode("Palace of Fine Arts, San Francisco, CA")

print("Oracle Park (origin):", origin_point)
print("Palace of Fine Arts (goal):", dest_point)
```

Oracle Park (origin): (37.7786059, -122.3894716)
Palace of Fine Arts (goal): (37.8029186, -122.4484029)

Step 2: Download the road network

Now we download the drivable roads for the area covering both places. Instead of hardcoding the bounding box, I calculate it from the two coordinates we got in Step 1, and add around half a mile of extra padding on all sides. The padding is needed because sometimes the shortest route goes slightly outside the direct rectangle between the two points, and we don't want to cut off the actual best route.

OSMnx gives us a directed graph (`MultiDiGraph`), so one-way streets are handled properly, same as a real GPS.

```
In [4]: #Build the bounding box from the two coordinates we got in Step 1.  
#We add some padding so the best route does not get cut off at the edges.  
pad = 0.01 #degrees, roughly 0.6 miles  
  
north = max(origin_point[0], dest_point[0]) + pad  
south = min(origin_point[0], dest_point[0]) - pad  
east = max(origin_point[1], dest_point[1]) + pad  
west = min(origin_point[1], dest_point[1]) - pad  
bbox = (west, south, east, north)  
print("Computed bbox (W, S, E, N):", bbox)  
  
#Download the drivable road network for this area  
G = ox.graph_from_bbox(bbox, network_type="drive")  
print("Number of nodes (intersections):", len(G.nodes))  
print("Number of edges (road segments):", len(G.edges))  
  
fig, ax = ox.plot_graph(G, node_size=2, edge_linewidth=0.4, figsize=(10, 8))
```

Computed bbox (W, S, E, N): (-122.45840290000001, 37.768605900000004, -122.37947159999999, 37.812918599999996)
Number of nodes (intersections): 2722
Number of edges (road segments): 7057



Step 3: Find the nearest graph nodes

The search algorithms work between nodes of the graph, but our two places may not be exactly on an intersection. So we find the nearest road network node for each location.

```
In [5]: orig_node = ox.distance.nearest_nodes(G, X=origin_point[1], Y=origin_point[0])
dest_node = ox.distance.nearest_nodes(G, X=dest_point[1], Y=dest_point[0])
print("Origin node:", orig_node, "| Destination node:", dest_node)
```

Origin node: 257316381 | Destination node: 2080819762

Step 4 — Algorithm 1: Dijkstra's algorithm (uninformed search)

Why I chose Dijkstra: It is the standard algorithm for shortest path when edge weights are not negative, and road lengths can never be negative. It is guaranteed to give the optimal (shortest) path, and it doesn't need any extra knowledge about the problem. So it is a good baseline to compare against. The only drawback is that it searches blindly in all directions from the start, so it wastes time exploring nodes which are far away from the destination.

```
In [6]: start = time.time()
dijkstra_route = nx.shortest_path(G, orig_node, dest_node, weight="length",
```

```

end = time.time()

dijkstra_len = nx.path_weight(G, dijkstra_route, weight="length")
dijkstra_miles = dijkstra_len / 1609.34

print("Dijkstra route has", len(dijkstra_route), "nodes")
print("Total driving distance:", round(dijkstra_len), "meters =", round(dijk
print("Time taken:", round((end - start) * 1000, 1), "ms")

```

Dijkstra route has 76 nodes
Total driving distance: 7577 meters = 4.71 miles
Time taken: 5.2 ms

Step 5 — Algorithm 2: A* search (informed search)

Why I chose A*: A* is an improvement over Dijkstra. Along with the distance travelled so far $g(n)$, it also uses a heuristic $h(n)$ which estimates how far the goal is. It always expands the path with smallest $f(n) = g(n) + h(n)$.

For the heuristic I used the **straight line (great-circle) distance** from the node to the destination. This heuristic is **admissible** — the actual road distance can never be less than the straight line distance, so $h(n)$ never overestimates. Because of this, A* is still guaranteed to give the optimal answer, but it searches in the direction of the goal instead of going everywhere. This is why real navigation systems use A* type algorithms.

In [7]: *#Heuristic: straight line distance (in meters) from node n1 to the goal node*
#Note: the distance formula itself comes from OSMnx (ox.distance.great_circle)
#We still need this small wrapper function because nx.astar_path calls the
#heuristic with node IDs, but great_circle needs lat/lon coordinates. So thi
#function just looks up the coordinates of each node and passes them on.

```

def straight_line_heuristic(n1, n2):
    return ox.distance.great_circle(G.nodes[n1]["y"], G.nodes[n1]["x"],
                                   G.nodes[n2]["y"], G.nodes[n2]["x"])

start = time.time()
astar_route = nx.astar_path(G, orig_node, dest_node,
                           heuristic=straight_line_heuristic, weight="length")
end = time.time()

astar_len = nx.path_weight(G, astar_route, weight="length")
astar_miles = astar_len / 1609.34

print("A* route has", len(astar_route), "nodes")
print("Total driving distance:", round(astar_len), "meters =", round(astar_n
print("Time taken:", round((end - start) * 1000, 1), "ms")

```

A* route has 76 nodes
Total driving distance: 7577 meters = 4.71 miles
Time taken: 11.6 ms

Step 6 — Extra comparison: BFS (unweighted search)

Just for comparison, I also ran plain BFS. BFS ignores the edge weights completely and only minimizes the number of road segments. This shows why we should not use unweighted search when edge costs are different from each other.

```
In [8]: #No weight given, so this is BFS on number of hops
bfs_route = nx.shortest_path(G, orig_node, dest_node)
bfs_len = nx.path_weight(G, bfs_route, weight="length")

print("BFS route has", len(bfs_route), "nodes (fewest road segments)")
print("Total driving distance:", round(bfs_len), "meters =", round(bfs_len /

extra = (bfs_len / dijkstra_len - 1) * 100
print("BFS route is", round(extra, 1), "% longer than the optimal route")
```

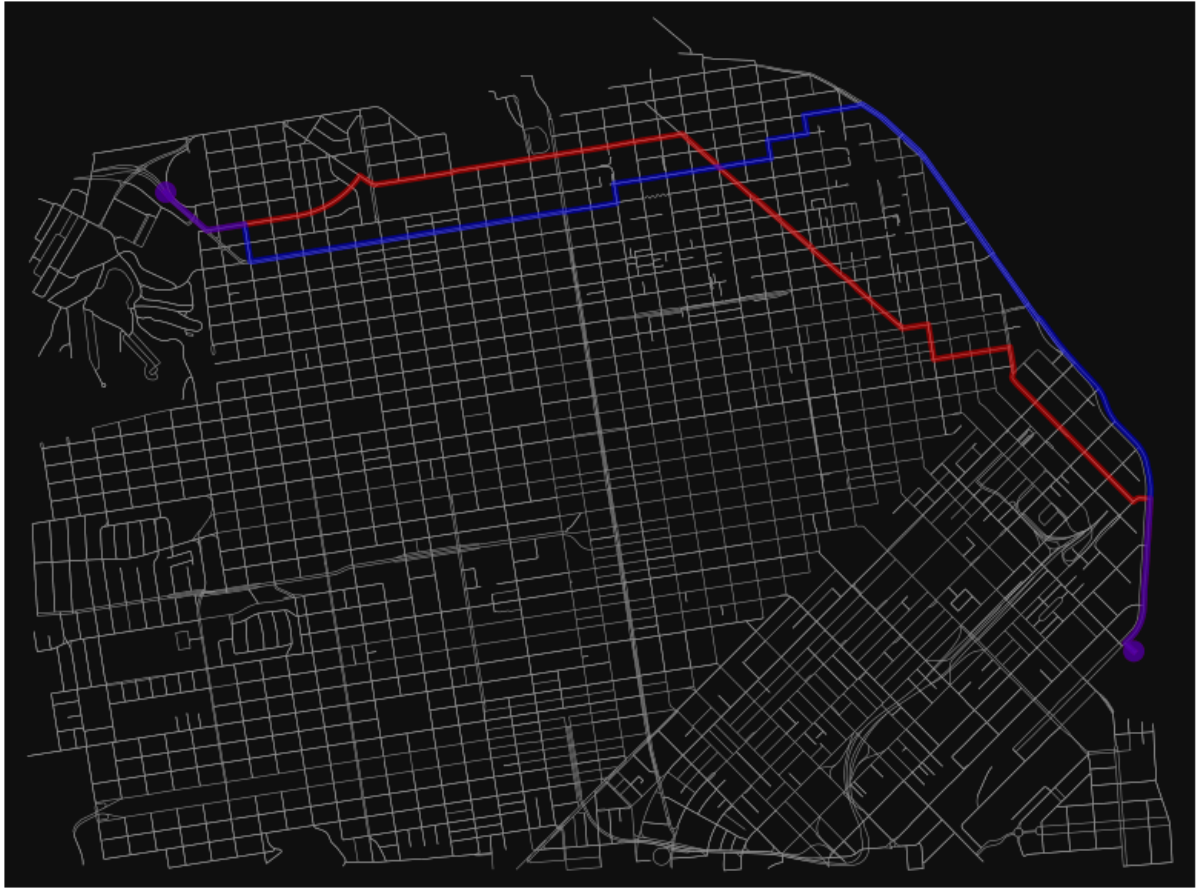
BFS route has 57 nodes (fewest road segments)
Total driving distance: 8176 meters = 5.08 miles
BFS route is 7.9 % longer than the optimal route

Step 7: Results and visualization

```
In [9]: print("Dijkstra distance:", round(dijkstra_len), "meters")
print("A* distance:         ", round(aster_len), "meters")
print("Both algorithms found the same route:", dijkstra_route == aster_route)

#Plot the optimal route (red) and the BFS route (blue)
fig, ax = ox.plot_graph_routes(G, routes=[dijkstra_route, bfs_route],
                               route_colors=["r", "b"], route_linewidths=3,
                               node_size=0, edge_linewidth=0.4, figsize=(10,
```

Dijkstra distance: 7577 meters
A* distance: 7577 meters
Both algorithms found the same route: True



```
In [10]: #Print the street names along the optimal route
print("Route summary (street, miles):")

prev_name = None
distance = 0
for i in range(len(dijkstra_route) - 1):
    u = dijkstra_route[i]
    v = dijkstra_route[i + 1]
    edge = G[u][v][0] #take the first edge between u and v
    name = edge.get("name", "(unnamed)")
    if isinstance(name, list):
        name = name[0]
    if name == prev_name:
        distance = distance + edge["length"]
    else:
        if prev_name is not None:
            print(" ", prev_name, "-", round(distance / 1609.34, 2), "miles")
            prev_name = name
            distance = edge["length"]
print(" ", prev_name, "-", round(distance / 1609.34, 2), "miles")
```

```
Route summary (street, miles):
  2nd Street - 0.04 miles
  King Street - 0.15 miles
  The Embarcadero - 0.37 miles
  Bryant Street - 0.06 miles
  Main Street - 0.61 miles
  Drumm Street - 0.09 miles
  Sacramento Street - 0.27 miles
  Sansome Street - 0.12 miles
  Washington Street - 0.09 miles
  Columbus Avenue - 1.01 miles
  Bay Street - 1.09 miles
  Cervantes Boulevard - 0.04 miles
  Alhambra Street - 0.26 miles
  Francisco Street - 0.32 miles
  Richardson Avenue - 0.06 miles
  Presidio Parkway - 0.13 miles
```

Discussion: Why these two algorithms?

I chose **Dijkstra** because it is the standard optimal algorithm for weighted graphs with non-negative weights, and our road network fits that perfectly. It needs no extra information, so it works as a reliable baseline. I chose **A*** because in a road network we get the node coordinates for free, so the straight line distance to the goal is an easy and admissible heuristic. A* keeps the same optimality guarantee as Dijkstra but explores much fewer useless nodes, since it moves towards the goal. Also, running two different algorithms is a good sanity check — both gave the exact same route and same distance, so I am confident the answer is correct. The BFS run shows the opposite case: it found a route with fewer segments, but the total distance was longer, which proves that ignoring weights gives a wrong idea of "shortest".

Part 2: How I would improve this solution

First, I would optimize for **travel time instead of distance**, because in real life nobody wants the shortest route, they want the fastest one. OSMnx has functions like `ox.add_edge_speeds` and `ox.add_edge_travel_times` which attach speed limits to each road and convert the edge weight from meters to seconds. The shortest route often goes through small slow streets, while a slightly longer route on a main road or highway is actually faster. If possible, I would also add **live traffic data** like Google Maps and Waze use, so the edge weights change with time — the best route at 8 AM on Monday should not be the same as the one at midnight.

Second, I would make the cost model more realistic at intersections. Right now my graph does not charge anything for turns, but in real driving, left turns across traffic, stop signs, and signals take extra time. I would add **turn penalties and intersection costs** (UPS actually does this in their ORION routing system — they avoid left turns).

OpenStreetMap also has extra data like number of lanes, road surface, and slope, which can be added into the edge weights to make the model closer to reality.

Third, I would improve the **heuristic**. The straight line heuristic is admissible but weak in a city like San Francisco, because the Bay, parks, and one-way streets force big detours, so the straight line estimate is too optimistic near the water. A better option is **landmark based heuristics (ALT)**, where we precompute actual shortest distances to a few landmark nodes and use the triangle inequality to get much tighter estimates. For very large maps, preprocessing methods like **contraction hierarchies** or dynamic programming tables (Poole & Mackworth section 3.8 discusses this) answer queries much faster. **Bidirectional search** — searching from both ends and meeting in the middle — is another standard speedup I would like to try.

Finally, I would think about what "best" route really means. A single cost number hides many trade-offs — fuel, safety, scenic value, or sending too much traffic through quiet residential streets (this social impact issue was discussed in our module readings also). Instead of one route, I would use **multi-objective search** and show the user a few options like fastest, shortest, and calmest, and let them choose. I would also handle re-planning when the driver misses a turn (algorithms like D* Lite do incremental re-planning), and compare my results with a real navigation API to see how far my simple model is from reality.

AI Usage Disclosure

I used Claude while working on this assignment. Specifically:

- In Part 2, the ideas of optimizing for **travel time instead of distance** and using **live traffic data** were my own. The remaining suggestions — turn penalties (UPS ORION), landmark-based heuristics (ALT), contraction hierarchies, D* Lite re-planning, and multi-objective search — came from the AI, and I reviewed them to understand what they mean before including them.
- I reviewed, ran, and verified all code and outputs myself - Just cross checked my solutions with Claude to make sure I wasn't missing anything.
- I used Grammarly to re-format my sentences as English is my second language.